Zapatero

Integración de tienda, back-office y automatización supervisada

Soluciones Informáticas para la Empresa · Tarea 2

Equipo: Natalia Castaño Lorente, Irene Almagro Andújar y Nikita Biriouk. Estado verificado: 4 de noviembre de 2026.

Informe de implementación y publicación. La tienda y el back-office están publicados en DonDominio; n8n está disponible en Render. Se han comprobado las APIs y el acceso desde Render a DonDominio. La ejecución conjunta de los tres escenarios con Telegram y un modelo real de IA se distingue expresamente de estas verificaciones.

Punto de partida y autoría

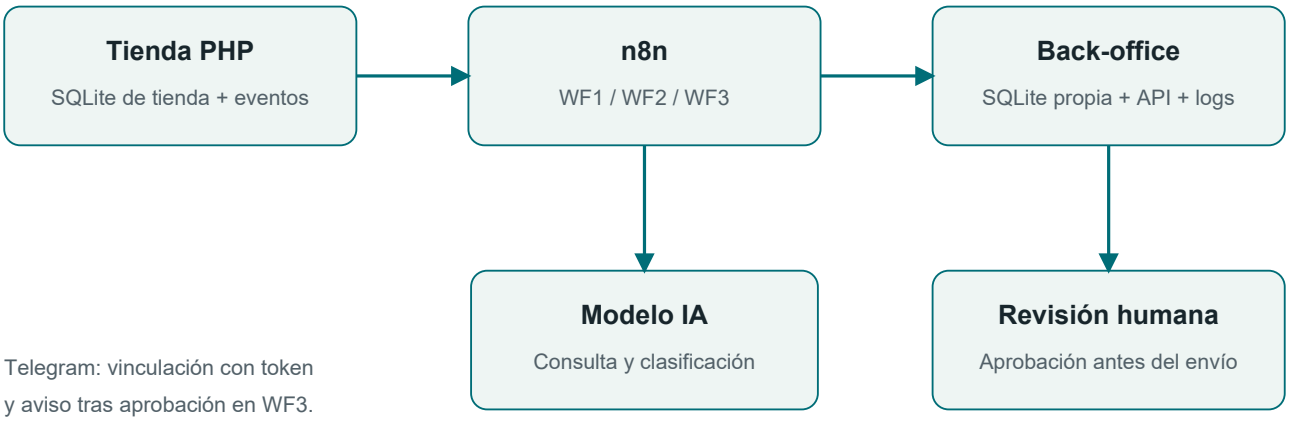
La base actual es https://github.com/HugoFernandezz/Zapatero, main en fe1307d, que integra la Tarea 2 de los compañeros. El ZIP inicial se utilizó para estudiar el proyecto; las mejoras se adaptaron después sobre esa base actualizada, conservando su historial, API de eventos y cambios de datos. No se presenta el proyecto como creado desde cero en esta intervención.

Integrante	Aportación identificada
Irene Almagro Andújar	Políticas y pie, retirada del contrato antiguo de Telegram, pruebas HTTP, publicación en Render y DonDominio, memoria y anexo con apoyo de IA
Natalia Castaño Lorente	Historial de tienda y proyecto M3, con integración y resolución de conflictos; commits c4109b4 y 286d993
Nikita Biriouk	Historial de API, back-office, agente, Telegram, métricas y workflows; commits 8e673ba, 5aa98f0 y 0526241
Equipo	Revisión del material, pruebas completas de escenarios, capturas complementarias y presentación conjunta

El reparto histórico se limita a lo identificable en Git y a lo declarado por Irene; no reconstruye todas las sesiones ni excluye otros autores del repositorio. La configuración de Render se publicó en codex/despliegue-n8n-irene. Los cambios de interfaz, seguridad y documentos se preparan sobre fe1307d en codex/irene-politicas-telegram, sin convertir el código previo en un commit propio.

1. Proceso integrado y arquitectura

Zapatero representa una tienda ficticia de zapatillas. El proceso comienza cuando una persona consulta un producto, elige talla, prepara el carrito y completa un pago simulado. La tienda registra un pedido y eventos de negocio. n8n consulta los eventos o recibe un webhook, valida el contrato y transforma la información al formato del back-office.



Telegram: vinculación con token y aviso tras aprobación en WF3.

La tienda utiliza PHP con Slim y una base SQLite. El back-office mantiene otra base, con pedidos normalizados, clientes, líneas, tickets, propuestas y registro de integración. Sus escrituras se realizan mediante una API autenticada; su interfaz se abre con una cuenta administradora. Ambos módulos comparten actualmente el despliegue PHP y la autenticación. La separación de almacenamiento y contrato existe, pero no hay aislamiento completo de infraestructura ni de credenciales por función.

WF1 sincroniza pedidos y cambios de estado. WF2 trata incidencias o solicitudes de soporte y genera una propuesta con IA o una derivación segura. WF3 vincula Telegram mediante un enlace de un solo uso y recoge las acciones que una persona ya ha aprobado.

Los tres escenarios obligatorios se corresponden con: un pedido que aparece en el back-office; un cambio operativo que actualiza su estado; y una incidencia asociada al pedido con seguimiento, revisión humana y aviso de Telegram. El código y los payloads están disponibles; la evidencia de su ejecución conjunta en el alojamiento público sigue pendiente.

El sitio muestra una franja académica y un pie con enlaces al carrito, al catálogo y a las secciones de las políticas. «Quiénes somos» estaba en el ZIP inicial, pero no figura en la base actual publicada por los compañeros y no se atribuye a esta mejora. La página /politicas explica pagos, envíos, soporte y uso de datos de prueba sin presentar la demostración como una actividad comercial.

2. Contrato de integración

La API de la tienda se encuentra en /api/v1 y permite leer eventos y pedidos con API_TOKEN. La del back-office se encuentra en /bo/api/v1 y utiliza un secreto distinto, BO_API_TOKEN. n8n envía el token en Authorization: Bearer; se admite X-Api-Token para alojamientos que eliminan Authorization. No se incluye ninguno de sus valores en esta memoria.

GET /api/v1/events admite after_id, limit y type. Responde con events, has_more y next_after_id. El consumidor conserva el cursor para continuar desde el último evento tratado. Los eventos incluyen event_id, schema_version, type, occurred_at y payload. Ejemplo de estructura de prueba:

```
{
  "event_id": "evt_11111111111111111111111111111111",
  "schema_version": 1,
  "type": "order.created",
  "occurred_at": "2026-11-03T10:00:00Z",
  "payload": {
    "order_code": "ZAP-20261103-0001",
    "total_cents": 12990,
    "items": 2
  }
}
```

El ejemplo describe el contrato; no es un resultado de una ejecución. Tras recibir el evento, el flujo consulta GET /api/v1/orders/{code}, que devuelve cliente de prueba, estado, líneas e importes en céntimos. POST /bo/api/v1/orders crea o actualiza la entidad. Los cambios se envían a /orders/{code}/status y los tickets a /tickets.

El contrato actual de Telegram acepta únicamente un objeto con token y chat_id como cadenas. El token tiene 32 caracteres hexadecimales, caduca a los 30 minutos y se guarda mediante su hash SHA-256. La variante antigua con order_code se ha retirado incluso para peticiones con Bearer válido.

La especificación extensa, incluidas respuestas y salida del agente, se conserva en docs/contrato-integracion.md. Los ejemplos de los tres escenarios están en n8n/payloads y deben rellenarse con códigos de pedidos realmente creados durante las pruebas.

3. Validaciones y controles

La integración valida la forma del identificador de evento, la versión del esquema, el tipo de evento y los campos obligatorios. Los pedidos usan el formato ZAP-AAAAMMDD-NNNN. El back-office exige importes enteros en céntimos, estados conocidos, correo con formato válido y líneas de pedido. Un valor como la cadena "100" no sustituye a un entero.

Las APIs utilizan una comparación de secretos mediante hash_equals. Si no se configura un token de al menos 24 caracteres, la API se desactiva con 503. Un token incorrecto o ausente devuelve 401. La interfaz administrativa exige sesión con rol administrador; los formularios modificadores usan CSRF. Las consultas de datos se realizan con sentencias preparadas y las vistas escapan el contenido mostrado.

Corrección del acceso antiguo de Telegram

La base del equipo ya implementaba enlaces de un solo uso, pero el controlador conservaba una alternativa que aceptaba order_code y chat_id. El Bearer demostraba que la petición conocía la credencial de integración; no demostraba acceso legítimo al pedido. La nueva versión elimina esa alternativa y el método del servicio que vinculaba directamente por código.

La prueba HTTP crea un pedido sincronizado, llama con Bearer correcto y payload antiguo, exige 422 y comprueba que el chat no cambia. También verifica que el enlace válido continúa funcionando, que su reutilización devuelve 410 y que tipos JSON incorrectos se rechazan sin provocar un error 500. Los rechazos autenticados quedan en el registro sin guardar el token en claro.

Políticas visibles

El pie compartido aparece en las páginas de tienda, cuenta y administración. Sus enlaces apuntan a secciones reales de /politicas. El contenido no promete entregas, recogidas ni reembolsos y distingue la compra ficticia de las conexiones externas que pueden producir avisos reales. La página se organiza con encabezados, navegación identificada y adaptación a pantallas pequeñas.

Estos controles se han comprobado localmente en las suites indicadas en la página 9. No constituyen una auditoría completa de seguridad ni una prueba de los servicios externos.

4. Errores, idempotencia y trazabilidad

El back-office registra resultado, origen, tipo de evento y referencias funcionales en bo_integration_log. Los workflows pueden incluir workflow y execution_id para relacionar la entrada con una ejecución de n8n. El registro se consulta en /bo/integracion; los eventos originales de la tienda permanecen disponibles en /admin/eventos.

Caso	Respuesta y efecto esperado
Payload no válido	422; no crea la entidad y deja rechazo
event_id repetido	duplicate; evita repetir los efectos de ese evento
Estado antiguo	stale; conserva el estado más reciente y lo registra
Pedido aún desconocido	pending_sync; reaplica el cambio cuando llega el pedido
Token Telegram desconocido	404; no vincula el chat
Token usado o caducado	410; impide su consumo
Pedido de Telegram sin sincronizar	409; mantiene el token para reintentar
Modelo sin configurar o sin salida válida	Propuesta segura para revisión manual

La idempotencia se apoya en event_id y en comprobaciones de entidades como ticket_ref. No equivale a una garantía universal de «exactamente una vez». Los servicios de red pueden fallar entre la acción externa y el registro de su resultado; se debe revisar el historial antes de repetir un envío.

El cursor de lectura reduce el reprocesamiento, pero no reemplaza la validación. La ejecución programada puede tardar hasta un minuto mientras n8n esté activo. Si el alojamiento lo suspende, no se ejecutan los sondeos hasta que el servicio vuelve a funcionar.

Para investigar un fallo: localizar el pedido o ticket, buscar el evento en la tienda, comprobar la ejecución del workflow, revisar el estado HTTP y consultar el registro del back-office. La captura debe mostrar identificadores de prueba y resultado, sin cabeceras de autenticación ni datos del chat.

Se incluyen payloads de error y de intento de manipular las instrucciones del agente. Son material para reproducir un caso, no evidencia de haberlo ejecutado contra un modelo real. Esa prueba debe documentarse conjuntamente con las demás capturas.

5. Agente de IA y herramientas

El componente funcional se encuentra en WF2, en el nodo «Agente de IA (bucle con herramientas)». Es un agente de soporte posventa que recibe el ticket, consulta datos del caso y propone una acción estructurada. Usa una API compatible con chat/completions y llamadas a herramientas. No se confunde con Codex o Claude Code, utilizados como ayuda durante el desarrollo.

Herramienta	Alcance y efecto
consultar_pedido	Solo el pedido asociado al ticket; recupera estado, líneas e incidencias
consultar_cliente	Solo el cliente obtenido al consultar ese pedido; recupera historial
clasificar_ticket	Asigna una vez categoría y prioridad al ticket actual; cambio reversible

El bucle permite hasta seis pasos y pide una herramienta por turno. Los argumentos se contrastan con el pedido, el cliente conocido y el ticket actual. Una herramienta fuera de la lista se rechaza. La clasificación está limitada a categorías y prioridades previstas.

Las instrucciones indican que el contenido del ticket no es fiable y no debe cambiar las reglas del agente. Prohíben cancelar pedidos, prometer reembolsos, pedir credenciales o compartir datos ajenos al caso. La salida se analiza como JSON y se valida antes de registrar una propuesta. La API del back-office vuelve a comprobar los campos admitidos.

La propuesta contiene tipo de caso, datos consultados, acción, justificación, confianza, prioridad, riesgos y modelo. Las acciones admitidas son mensaje al cliente, propuesta de estado, tarea interna o no actuar. Un estado prohibido se deriva a revisión; mensajes que aluden a dinero o cancelaciones generan advertencias y reducen la confianza. Estos filtros no eliminan todos los riesgos de una respuesta incorrecta.

Si faltan clave, URL o modelo, si la API falla o no hay una respuesta válida dentro del límite, se produce una tarea interna de alta prioridad con confianza cero. Este modo seguro no demuestra que se haya utilizado un modelo ni que se hayan llamado herramientas con éxito. El nombre efectivo del modelo debe obtenerse de la configuración y de una ejecución real.

6. Supervisión humana y comunicación

Todas las propuestas se crean como pending_review. Una persona administradora las revisa en /bo/agente y decide aprobar o rechazar, con su identidad y comentario. El servicio valida la transición para que una decisión ya tomada no se cambie por esa misma operación. WF3 recoge únicamente acciones aprobadas.

El resultado de ejecución puede ser executed, executed_simulated o execution_failed. La API rechaza el informe de ejecución de una propuesta que no ha sido aprobada. El rechazo humano impide ejecutar esa propuesta por el camino previsto. Las propuestas de cambio de estado y las tareas internas no constituyen una automatización general de cambios de la tienda; hay que comprobar su tratamiento concreto en WF3.

Vinculación del chat

Desde la página del pedido, una petición con comprobación de acceso y CSRF emite un enlace t.me con un secreto aleatorio. Telegram recibe /start TOKEN y WF3 lo envía a la API del back-office junto con el chat_id. La vinculación consume el token dentro de una transacción. Emitir otro invalida el anterior para ese pedido.

Un código ZAP no es suficiente para vincular Telegram. El enlace de un solo uso debe tratarse como un secreto durante sus 30 minutos de vigencia: no se comparte en capturas ni se usa como enlace público. La asociación se guarda en el cliente del back-office, por lo que las pruebas deben comprobar el comportamiento con varios pedidos del mismo cliente.

Datos y riesgos

La cuenta de Telegram puede corresponder a una persona real aunque el pedido sea ficticio. Si el modelo es externo, puede recibir el contenido del ticket y datos devueltos por las herramientas. El código minimiza algunos datos de la consulta de cliente, pero esa minimización no implica que todas las respuestas de pedido estén anonimizadas. Deben utilizarse solo datos de prueba y revisar las salidas antes de aprobar.

Las respuestas de ayuda o vinculación del bot son operaciones técnicas; la revisión humana descrita se refiere a la propuesta de respuesta del agente al cliente. La entrega debe mostrar una propuesta pendiente, la decisión humana y el resultado posterior, con los secretos ocultos.

7. Métricas y decisión de implantación

MetricsService calcula indicadores a partir de las tablas de la tienda y del back-office. No introduce cifras manuales. La interfaz /bo/metricas incluye gráficos SVG y expone sus datos en /bo/metricas/datos; la API autenticada ofrece /bo/api/v1/metrics. Los denominadores sin datos producen ausencia de valor en lugar de una tasa inventada.

Indicador	Base de cálculo y utilidad
Pedidos e importe	Recuento de bo_orders; suma de céntimos sin cancelados
Productos y estados	Cantidades por producto y pedidos agrupados por estado
Tickets abiertos	open e in_progress; carga de atención pendiente
Resolución simulada	Media entre creación y resolución de tickets cerrados
Integración y errores	Resultados y error_code en bo_integration_log
Conversión del recorrido	Sesiones con pedido respecto a sesiones con producto visto
Propuestas y decisiones	Recuento y aprobación sobre propuestas decididas

La tasa de conversión usa una definición propia por sesión; no demuestra atribución causal ni equivale a una herramienta analítica comercial. El tiempo de resolución refleja las marcas del ejercicio, no un compromiso de servicio. Una confianza alta del modelo tampoco garantiza corrección.

La justificación ampliada y las referencias bibliográficas figuran en docs/metricas-justificacion.md. Las suites comprueban los cálculos con registros generados durante la prueba. Esos datos controlados no deben confundirse con resultados del entorno público.

Comparación de alternativas

La matriz recibida compara aplicación propia con n8n, ERP/CRM parametrizable y SaaS de comercio con automatización. Pondera doce criterios, entre ellos funcionalidad, coste, esfuerzo, seguridad, integración, mantenimiento y dependencia del proveedor. Sus resultados son 4,06; 3,47 y 3,35 sobre cinco, respectivamente.

Se recomienda la solución propia con n8n para el alcance académico porque permite observar contratos, errores y supervisión. Las puntuaciones son juicios de diseño del equipo, no mediciones de mercado ni presupuestos. Los servicios gestionados pueden mejorar operación o escalabilidad y requieren otra evaluación si el caso pasa a actividad comercial. La tabla completa está en docs/matriz-decision.md.

8. Publicación y verificación

La tienda y el back-office están publicados en https://www.zapatero.onl y https://www.zapatero.onl/bo. Se guardó una copia privada completa de 395 entradas del hosting, se comprobó la migración sobre una copia de su SQLite y se subieron los archivos de ejecución necesarios sobre la versión fe1307d. Se conservaron storage, vendor, la cuenta administradora y la configuración de correo. No se ejecutó db:init en el servidor.

n8n 2.46.4 está en https://zapatero-n8n.onrender.com, con servicio Free zapatero-n8n y PostgreSQL Free zapatero-n8n-db. Las URLs y los dos secretos de integración están configurados en los alojamientos, fuera de Git. Los tres workflows del equipo están importados y guardados; siguen sin publicar para no activar sondeos o envíos antes de la demostración conjunta y de configurar IA y Telegram.

Se corrigieron un HTTP 429 del registro inicial usando Docker Hub oficial y fallos de arranque ajustando memoria, concurrencia, siete tipos de nodo y TTL del runner. El arranque configura HTTPS, cookie segura y proxy. La instancia Free puede suspenderse por inactividad y la base Free caduca; no se ha contratado ningún plan de pago.

Comprobación	Resultado documentado
Cuatro suites PHP sobre fe1307d y mejoras	122 comprobaciones correctas
Tienda, políticas y acceso de n8n	HTTP 200
Back-office sin sesión / acceso con cuenta existente	Redirección al login / interfaz correcta
API de back-office sin token / con token	HTTP 401 / 200
Contrato antiguo de Telegram con Bearer válido	HTTP 422 en servidor público
Render → salud de tienda y API del back-office	Ejecución n8n #1 correcta; ambos HTTP 200
Tres escenarios con modelo y Telegram reales	Sin ejecutar en esta intervención

El workflow de verificación 7kUldPi4bl3J10JS finalizó el 04/11/2026 a las 05:58:46 UTC, en 4,127 s. Solo consulta salud y métricas: acredita conectividad desde otra red, no sincronización de pedidos ni envío de mensajes. El JSON y la captura se guardan en docs/evidencias junto con el informe de siete comprobaciones públicas y la prueba del contrato retirado.

Los estilos se versionan por su contenido para evitar la caché de siete días observada en el hosting. Commits de Render: 522833e, ce39b88, 97199dc y 9dbfdaf. scripts/comprobar-publicacion.ps1 reproduce las comprobaciones de lectura sin mostrar secretos.

9. Limitaciones y entrega

La aplicación sigue siendo un prototipo sin ventas, cobros ni entregas reales. SQLite limita las escrituras concurrentes. Tienda y back-office comparten infraestructura y autenticación, aunque sus bases y contratos estén separados. No se afirma independencia física de los servicios PHP.

La publicación y la conectividad externa están comprobadas. Los tres workflows funcionales se han importado, pero no se han publicado. No se han aportado credenciales del proveedor de IA ni del bot de Telegram a esta instancia. Por ello no se afirma una ejecución real del modelo, una respuesta aprobada enviada al chat ni los tres escenarios públicos completos. Son pruebas conjuntas del equipo antes de defender esa parte funcional.

Las capturas y la presentación deben prepararse conjuntamente. El guion docs/pruebas-y-defensa.md detalla el pedido ficticio, el cambio operativo, la incidencia, la revisión humana y el aviso que hay que mostrar. La verificación desde Render ya acredita otra red; no se afirma haber realizado además una prueba móvil con Wi-Fi desactivado.

Mejoras futuras

Separar infraestructura y permisos de cada módulo cuando el alcance lo justifique; usar credenciales administrativas privadas para una instalación permanente; verificar copias y recuperación; limitar intentos de acceso; minimizar datos enviados al modelo y ensayar fallos de red y reintentos. Estos puntos no se presentan como implantados.

Los filtros y el prompt no eliminan los errores de IA. Un envío de Telegram puede fallar entre la acción externa y el registro; hay que revisar el historial antes de repetirlo. El token Bearer del back-office cubre varias funciones y admite una separación posterior por herramienta.

Fuentes y material de revisión

El contenido técnico se contrasta con el código de la base fe1307d, sus workflows y las pruebas guardadas. La matriz y la justificación de métricas originales se conservan en el repositorio. El anexo de IA declara las herramientas utilizadas y los datos no registrados, sin inventar trazabilidad.

- Render, publicación de n8n: https://render.com/docs/deploy-n8n
- Render, configuración: https://render.com/docs/blueprint-spec
- Render, entorno: https://render.com/docs/environment-variables
- n8n, nodos: https://docs.n8n.io/hosting/configuration/environment-variables/nodes/
- n8n, runners: https://docs.n8n.io/hosting/configuration/environment-variables/task-runners/
- DonDominio, FTP: https://www.dondominio.com/es/help/120/como-subo-web-mediante-ftp/

Configuración contrastada el 3 y el 4 de noviembre de 2026. La entrega conserva fuentes editables, PDF y evidencias sin credenciales. El equipo debe revisar el contenido y explicar las aportaciones que presenta.